Booking Scraper Initial Investigation

Project Brief and Explanation

The Thesis

Target assets

- Small hotels across Italy, bought directly from the owner.
- Must be multi-unit.

Pricing inefficiency (the opportunity)

- Most Italian hotels currently run *no dynamic pricing.*
- Clear room to revenue-maximise by moving rates with demand (seasonality, events, competitor rates).
- First, understand *why* each hotel is priced the way it is - then enable AI to replicate human revenue management teams.
- 77% independent ownership (inefficient ops and no reason to optimise) - limited succession plans.

Value Creation Levers

1. Revenue optimisation

- Introduce dynamic rate management tied to demand and events.
- Introduce new layer of algorithmic pricing striping out traditional revenue management

2. Operational cost-out

- Strip out staff via IoT: electric/coded locks, automated check-in, etc.
- Centralise shared resources across units (achievable even without AI).

3. Structure

- Sign either to *own* the hotel or to run *operations only*.

Economics / Distribution Costs

- OTA commission booking (dot) com: ~17–18% of booking value.
- Corporate agency: ~10-15%, but this figure hides other unaccounted expenses.

Rate scraping (competitive intelligence)

- Scraping every rate, per room type, over the next ~12 months.
- Works around Booking.com's API rather than using it directly.

- Runs in batches of 60 days to avoid being blocked.
- Slow process as a result.

Key Definitions

- ADR – Average Daily Rate.
- RevPAR – Revenue Per Available Room.
- Occupancy – how full the hotel is.
- Supply – number of room nights available in the area to sell in a given month.

Open Questions / To Be Answered

- Target return / exit: *[TBD]* — what's the target IRR or multiple, and what's the exit route?
- Capital per unit: *[TBD]* — how much capital is required to acquire and/or convert each unit?
- Deal sourcing at scale: *[TBD]* — how do we find and win these owner-direct deals repeatably?
- Own vs. operate: *[TBD]* — which structure is preferred, and under what conditions do we pick each?
- Other distribution expenses: *[TBD]* — what are the hidden costs behind the ~15% corporate agency figure?
- Scraping viability: *[TBD]* — is working around Booking.com's API sustainable / compliant long-term?
- Revenue uplift estimate: *[TBD]* — how much RevPAR gain do we expect from introducing dynamic pricing?
- Ops savings estimate: *[TBD]* — what staff cost reduction is realistic from the IoT/centralisation play?

Project Code

```
# booking_property_chunked_any_vfix.py
# Booking.com property scraper (ANY hotel) — chunked, nav-safe, dual layout, live progress
# Fixes:
# - Uses compiled regex correctly (no flags on search)
# - Normalizes property slug (strips ".en-gb.html"/".html") for robust off-property guard & tidy filenames

import asyncio, re, time, sys
from dataclasses import dataclass, asdict
from datetime import date, timedelta, datetime
from typing import List, Optional, Tuple, Iterable
from pathlib import Path
import pandas as pd
from playwright.async_api import async_playwright, TimeoutError as PWTimeout, Error as PWError

# ----- make PyCharm show prints immediately -----
try:
    sys.stdout.reconfigure(line_buffering=True)  # Python 3.7+
except Exception:
    pass

# ---------------- CONFIG (EDIT THESE) ----------------
PROPERTY_URL     = "https://www.booking.com/hotel/ch/base-geneva.en-gb.html"  # <-- change to any property
ADULTS           = 2
NIGHTS           = 3
DATE_MODE        = "daily"       # 'daily' or 'weekends'
```

```
HORIZON_DAYS     = 365            # how far ahead from today (e.g., 365)

HEADLESS         = True
TIMEOUT_MS       = 24000

# Concurrency & chunking (prevents slowdowns/throttling on long runs)
MAX_CONCURRENCY  = 5             # 4-7 is usually safe
CHUNK_SIZE       = 60            # restart browser/context every N dates
COOLDOWN_BETWEEN_CHUNKS = 6      # seconds pause between chunks

# Progress & debugging
PROGRESS_EVERY   = 5             # print every N completed dates
VERBOSE          = False         # set True for per-date step logs
SAVE_DEBUG_SHOTS = False         # True saves screenshots on "no content"
DEBUG_DIR        = Path("debug_shots")

# ---------------- SELECTORS / REGEX ----------------
ROOM_LIST_CONTAINERS = "[data-testid='room-list'], [data-component='RoomCardsList']"
ROOM_CARD_SELECTOR   = "[data-testid='room-card']"
TITLE_SELECTOR       = "[data-testid='room-card-title']"
PRICE_PRIMARY        = "[data-testid='price-and-discounted-price']"

HPRT_FORM            = "#hprt-form"
HPRT_ROW_SELECTOR    = "#hprt-form tr.hprt-table-tr, #hprt-form tr.js-rt-block-row, #hprt-form tr.hprt-table-cheapest"
HPRT_NAME_SELECTOR   = ".hprt-roomtype-link, .hprt-roomtype-name, .hprt-roomtype-icon-link"
HPRT_PRICE_SELECTOR  = ".hprt-price-price-standard, .hprt-price, .bui-price-display__value"

# compiled regexes (DON'T pass flags again when using .search)
SIMILAR_BLOCK_RE     = re.compile(r"(similar properties|explore other options|properties nearby)", re.I)
CAPTCHA_RE           = re.compile(r"(verify you|security check|are you a robot)", re.I)
GBP_RE               = re.compile(r"£¥s*([¥d,]+(?:¥.¥d{2})?)")

# ---------------- UTILS ----------------
def log(msg: str):
    if VERBOSE:
        ts = datetime.now().strftime("%H:%M:%S")
        print(f"[{ts}] {msg}", flush=True)

def extract_property_slug(url: str) -> str:
    """Grab the property's slug from its Booking URL and normalize it (strip locale/html)."""
    m = re.search(r"/hotel/[^/]+/([^/?#]+)", url.lower())
    slug = m.group(1) if m else ""
    # strip ".en-gb.html" or any ".<locale>.html" or plain ".html"
    slug = re.sub(r"¥.(?:[a-z]{2}-[a-z]{2})¥.html$|¥.html$", "", slug)
    return slug

PROPERTY_SLUG = extract_property_slug(PROPERTY_URL)
print(f"Target property slug: {PROPERTY_SLUG}", flush=True)

# Name exports with the slug so multiple runs/properties don't collide
OUTPUT_XLSX = f"Booking_{PROPERTY_SLUG or 'property'}_{NIGHTS}N_chunked.xlsx"
OUTPUT_CSV  = f"Booking_{PROPERTY_SLUG or 'property'}_{NIGHTS}N_chunked.csv"

def booking_url(cin: date, nights: int, adults: int) -> str:
    cout = cin + timedelta(days=nights)
    return (
        f"{PROPERTY_URL}"
```

```
        f"?checkin={cin:%Y-%m-%d}&checkout={cout:%Y-%m-%d}"
        f"&group_adults={adults}&no_rooms=1&group_children=0&sb_price_type=total"
        f"&lang=en-gb&selected_currency=GBP"
    )

def generate_dates(mode: str, start: date, days: int) -> List[date]:
    if mode == "daily":
        return [start + timedelta(d) for d in range(days + 1)]
    # weekends: next Friday + next Sunday up to horizon
    out: List[date] = []
    next_fri = start + timedelta((4 - start.weekday()) % 7)
    next_sun = start + timedelta((6 - start.weekday()) % 7)
    for i in range(60):  # ~60 weeks cap
        for d in (next_fri + timedelta(weeks=i), next_sun + timedelta(weeks=i)):
            if (d - start).days <= days:
                out.append(d)
    return out

def chunked(seq: List[date], size: int) -> Iterable[List[date]]:
    for i in range(0, len(seq), size):
        yield seq[i:i+size]

def parse_total_robust(raw_text: str, nights: int) -> Tuple[Optional[float], Optional[float], str]:
    """Take the MAX £ value in the price block as the total for the stay (Booking shows totals)."""
    raw = " ".join((raw_text or "").split())
    nums = [float(x.replace(",", "")) for x in GBP_RE.findall(raw)]
    if not nums:
        return (None, None, raw)
    total = max(nums)
    nightly = round(total / nights, 2) if nights else None
    return (round(total, 2), nightly, raw)

# ---------------- DATA MODEL ----------------
@dataclass
class QuoteRow:
    check_in: str
    check_out: str
    nights: int
    room_name_raw: str        # Booking's exact room name
    status: str               # ok | unavailable | no_price_found | no_cards | redirected | captcha | timeout
    price_text: str
    total_gbp: float | None
    nightly_gbp: float | None
    url: str
    captured_at: str
    source: str = "Booking"

# ---------------- PAGE HELPERS ----------------
async def accept_cookies(page):
    for sel in ("#onetrust-accept-btn-handler", "button:has-text('Accept all')", "button:has-text('Accept')"):
        try:
            if await page.is_visible(sel, timeout=1200):
                await page.click(sel); break
        except Exception:
            pass

async def open_calendar(page):
```

```
    for sel in (
        "[data-testid='date-display-field-start']",
        "button:has([data-testid='date-display-field-start'])",
        "button:has-text('Check availability')",
        "button:has-text('See availability')",
    ):
        try:
            el = page.locator(sel).first
            if await el.count():
                await el.click(); await page.wait_for_timeout(200); return True
        except Exception:
            pass
    return False

async def click_calendar_day(page, date_str: str) -> bool:
    # click a calendar day; some layouts auto-navigate after the 2nd click
    for _ in range(24):  # up to 24 months forward
        el = page.locator(f"[data-date='{date_str}'], span[data-date='{date_str}']").first
        try:
            if await el.count():
                await el.click(no_wait_after=True)
                return True
        except PWError:
            return True
        # next-month arrows
        for nxt in ("button[aria-label='Next month']",
                    "[data-bui-ref='calendar-next']",
                    "button[data-testid='date-picker-calendar-right-arrow']"):
            try:
                if await page.locator(nxt).count():
                    await page.locator(nxt).first.click()
                    await page.wait_for_timeout(120)
                    break
            except Exception:
                pass
    return False

async def apply_dates_safe(page, cin: date, cout: date):
    log(f"Applying dates {cin:%Y-%m-%d} → {cout:%Y-%m-%d}")
    await open_calendar(page)
    await click_calendar_day(page, f"{cin:%Y-%m-%d}")
    await page.wait_for_timeout(150)
    try:
        async with page.expect_navigation(wait_until="domcontentloaded", timeout=7000):
            await click_calendar_day(page, f"{cout:%Y-%m-%d}")
        log("Auto-navigation after selecting checkout")
    except PWTimeout:
        # click submit to trigger nav if auto-nav didn't happen
        for sel in ("button[type='submit']",
                    "button:has-text('Check availability')",
                    "button:has-text('See availability')",
                    "button:has-text('Show prices')"):
            btn = page.locator(sel).first
            if await btn.count():
                try:
                    async with page.expect_navigation(wait_until="domcontentloaded", timeout=7000):
                        await btn.click()
```

```
                    log("Navigated after pressing submit")
                    break
                except PWTimeout:
                    await page.wait_for_load_state("domcontentloaded", timeout=5000)
                    log("No navigation; DOMContentLoaded reached")
                    break

async def trigger_lazy(page):
    # encourage room list/table to render
    for _ in range(4):
        try:
            await page.evaluate("window.scrollBy(0, document.body.scrollHeight)")
        except Exception:
            pass
        await page.wait_for_timeout(220)

async def find_room_cards(page):
    # scope to property room list first; then fallback to any 'room-card' with a title
    room_list = page.locator(ROOM_LIST_CONTAINERS).first
    if await room_list.count():
        cards = room_list.locator(ROOM_CARD_SELECTOR).filter(has=room_list.locator(TITLE_SELECTOR))
        if await cards.count():
            return cards
    return page.locator(ROOM_CARD_SELECTOR).filter(has=page.locator(TITLE_SELECTOR))

# ---------------- SCRAPE ONE DATE ----------------
async def scrape_once(context, cin: date) -> List[QuoteRow]:
    cout = cin + timedelta(days=NIGHTS)
    url = booking_url(cin, NIGHTS, ADULTS)
    rows: List[QuoteRow] = []
    page = await context.new_page()
    try:
        log(f"{cin:%Y-%m-%d} open url")
        await page.goto(url, wait_until="domcontentloaded", timeout=TIMEOUT_MS)
        await accept_cookies(page)

        # Off-property/redirect or captcha?
        body_lower = (await page.content()).lower()
        if ("/searchresults" in page.url
            or "/hotel/" not in page.url
            or (PROPERTY_SLUG and PROPERTY_SLUG not in page.url.lower())):
            rows.append(QuoteRow(f"{cin:%Y-%m-%d}", f"{cout:%Y-%m-%d}", NIGHTS,
                                 "(redirected to searchresults)", "redirected",
                                 "", None, None, page.url, datetime.now().strftime("%Y-%m-%d %H:%M:%S")))
            await page.close(); return rows
        if CAPTCHA_RE.search(body_lower):
            rows.append(QuoteRow(f"{cin:%Y-%m-%d}", f"{cout:%Y-%m-%d}", NIGHTS,
                                 "(captcha)", "captcha",
                                 "", None, None, page.url, datetime.now().strftime("%Y-%m-%d %H:%M:%S")))
            await page.close(); return rows

        # Apply dates via datepicker if the URL params were ignored
        await apply_dates_safe(page, cin, cout)
        await trigger_lazy(page)

        # A) Modern room-cards
        cards = await find_room_cards(page)
```

```
        if await cards.count() > 0:
            count = await cards.count()
            log(f"{cin:%Y-%m-%d} found {count} room-cards")
            for i in range(count):
                card = cards.nth(i)
                title_el = card.locator(TITLE_SELECTOR).first
                if not await title_el.count():
                    continue
                room_name = (await title_el.inner_text()).strip()

                # sold out?
                if await card.locator("text=/sold out|unavailable|fully booked|not available/i").count() > 0:
                    rows.append(QuoteRow(f"{cin:%Y-%m-%d}", f"{cout:%Y-%m-%d}", NIGHTS,
                                         room_name, "unavailable", "", None, None, page.url,
                                         datetime.now().strftime("%Y-%m-%d %H:%M:%S")))
                    continue

                price_el = card.locator(PRICE_PRIMARY).first
                if not await price_el.count():
                    price_el = card.locator("span:has-text('£')").first
                if await price_el.count():
                    raw = (await price_el.inner_text()).strip()
                    total, nightly, cleaned = parse_total_robust(raw, NIGHTS)
                    if total is not None:
                        rows.append(QuoteRow(f"{cin:%Y-%m-%d}", f"{cout:%Y-%m-%d}", NIGHTS,
                                             room_name, "ok", cleaned, total, nightly, page.url,
                                             datetime.now().strftime("%Y-%m-%d %H:%M:%S")))
                        continue

                rows.append(QuoteRow(f"{cin:%Y-%m-%d}", f"{cout:%Y-%m-%d}", NIGHTS,
                                     room_name, "no_price_found", "", None, None, page.url,
                                     datetime.now().strftime("%Y-%m-%d %H:%M:%S")))
            await page.close(); return rows

        # B) Legacy HPRT table
        hprt = page.locator(HPRT_FORM)
        if await hprt.count() > 0:
            rows_sel = page.locator(HPRT_ROW_SELECTOR)
            rcount = await rows_sel.count()
            log(f"{cin:%Y-%m-%d} HPRT rows {rcount}")
            if rcount == 0:
                if SAVE_DEBUG_SHOTS:
                    DEBUG_DIR.mkdir(parents=True, exist_ok=True)
                    await page.screenshot(path=str(DEBUG_DIR / f"{cin:%Y-%m-%d}_hprt_empty.png"), full_page=True)
                rows.append(QuoteRow(f"{cin:%Y-%m-%d}", f"{cout:%Y-%m-%d}", NIGHTS,
                                     "(hprt present but empty)", "no_cards", "", None, None, page.url,
                                     datetime.now().strftime("%Y-%m-%d %H:%M:%S")))
                await page.close(); return rows

            for i in range(rcount):
                row = rows_sel.nth(i)
                name_el = row.locator(HPRT_NAME_SELECTOR).first
                room_name = (await name_el.inner_text()).strip() if await name_el.count() else "(no title)"

                if await row.locator("text=/sold out|unavailable|fully booked|not available/i").count() > 0:
                    rows.append(QuoteRow(f"{cin:%Y-%m-%d}", f"{cout:%Y-%m-%d}", NIGHTS,
                                         room_name, "unavailable", "", None, None, page.url,
```

```
                                        datetime.now().strftime("%Y-%m-%d %H:%M:%S")))
                    continue

                price_el = row.locator(HPRT_PRICE_SELECTOR).first
                if not await price_el.count():
                    price_el = row.locator("span:has-text('£')").first
                if await price_el.count():
                    raw = (await price_el.inner_text()).strip()
                    total, nightly, cleaned = parse_total_robust(raw, NIGHTS)
                    if total is not None:
                        rows.append(QuoteRow(f"{cin:%Y-%m-%d}", f"{cout:%Y-%m-%d}", NIGHTS,
                                             room_name, "ok", cleaned, total, nightly, page.url,
                                             datetime.now().strftime("%Y-%m-%d %H:%M:%S")))
                        continue

                rows.append(QuoteRow(f"{cin:%Y-%m-%d}", f"{cout:%Y-%m-%d}", NIGHTS,
                                     room_name, "no_price_found", "", None, None, page.url,
                                     datetime.now().strftime("%Y-%m-%d %H:%M:%S")))
            await page.close(); return rows

        # No cards/table: if "similar properties" block exists, mark unavailable; else no_cards
        content = await page.content()
        status = "unavailable" if SIMILAR_BLOCK_RE.search(content) else "no_cards"
        if SAVE_DEBUG_SHOTS:
            DEBUG_DIR.mkdir(parents=True, exist_ok=True)
            await page.screenshot(path=str(DEBUG_DIR / f"{cin:%Y-%m-%d}_{status}.png"), full_page=True)
        rows.append(QuoteRow(f"{cin:%Y-%m-%d}", f"{cout:%Y-%m-%d}", NIGHTS,
                             f"({status})", status, "", None, None, page.url,
                             datetime.now().strftime("%Y-%m-%d %H:%M:%S")))
        await page.close(); return rows

    except PWTimeout:
        await page.close()
        return [QuoteRow(f"{cin:%Y-%m-%d}", f"{cout:%Y-%m-%d}", NIGHTS,
                         "(timeout)", "timeout", "", None, None, url,
                         datetime.now().strftime("%Y-%m-%d %H:%M:%S"))]

async def scrape_one_date(context, cin: date) -> List[QuoteRow]:
    # Retry once on Playwright-level races
    try:
        return await scrape_once(context, cin)
    except PWError:
        log(f"{cin:%Y-%m-%d} retrying after Playwright error")
        return await scrape_once(context, cin)

# ---------------- MAIN (chunked) ----------------
async def run():
    start = date.today()
    all_dates = generate_dates(DATE_MODE, start, HORIZON_DAYS)
    total = len(all_dates)
    print(f"Starting {total} dates | mode={DATE_MODE}, nights={NIGHTS}, adults={ADULTS} | property={PROPERTY_SLUG}",
flush=True)

    global_done = 0
    all_rows: List[QuoteRow] = []

    async with async_playwright() as p:
```

```
        chunk_idx = 0
        for batch in chunked(all_dates, CHUNK_SIZE):
            chunk_idx += 1
            print(f"¥n--- Chunk {chunk_idx}: {len(batch)} dates ---", flush=True)

            # fresh browser/context per chunk
            browser = await p.chromium.launch(headless=HEADLESS)
            context = await browser.new_context(
                locale="en-GB",
                timezone_id="Europe/London",
                user_agent=("Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                            "AppleWebKit/537.36 (KHTML, like Gecko) "
                            "Chrome/124.0.0.0 Safari/537.36"),
                viewport={"width": 1280, "height": 900},
            )

            # Block heavy assets (keep CSS/JS/XHR)
            async def route_handler(route):
                if route.request.resource_type in {"image", "media", "font"}:
                    return await route.abort()
                return await route.continue_()
            await context.route("**/*", route_handler)

            sem = asyncio.Semaphore(MAX_CONCURRENCY)
            async def task(d):
                async with sem:
                    return await scrape_one_date(context, d)

            tasks = [asyncio.create_task(task(d)) for d in batch]
            start_ts = time.time()

            for fut in asyncio.as_completed(tasks):
                chunk_rows = await fut
                all_rows.extend(chunk_rows)
                global_done += 1
                if (global_done % PROGRESS_EVERY == 0) or (global_done == total):
                    elapsed = int(time.time() - start_ts)
                    print(f"Progress: {global_done}/{total} dates  -  chunk elapsed {elapsed}s", flush=True)

            await browser.close()

            if global_done < total and COOLDOWN_BETWEEN_CHUNKS > 0:
                print(f"Cooling down {COOLDOWN_BETWEEN_CHUNKS}s before next chunk…", flush=True)
                await asyncio.sleep(COOLDOWN_BETWEEN_CHUNKS)

    # Export
    df = pd.DataFrame([asdict(r) for r in all_rows])
    room_names = (df.loc[df["room_name_raw"] != "", ["room_name_raw"]]
                    .drop_duplicates()
                    .sort_values("room_name_raw")
                    .reset_index(drop=True))

    cols = ["check_in","check_out","nights","room_name_raw","status",
            "price_text","total_gbp","nightly_gbp","url","captured_at","source"]
    if not df.empty:
        df = df[cols].sort_values(["check_in","room_name_raw","status"])
    else:
```

```
        df = pd.DataFrame(columns=cols)

    with pd.ExcelWriter(OUTPUT_XLSX, engine="openpyxl") as writer:
        df.to_excel(writer, sheet_name="Quotes", index=False)
        room_names.to_excel(writer, sheet_name="RoomNames", index=False)

    df.to_csv(OUTPUT_CSV, index=False)
    print(f"¥nSaved: {OUTPUT_XLSX} and {OUTPUT_CSV}", flush=True)
    print(df.groupby("status").size() if not df.empty else "No rows collected.", flush=True)

if __name__ == "__main__":
    asyncio.run(run())
```